Python
for
Data Science, AI & Development

COMPREHENSIVE LEARNING NOTES

Data Collection • Data Analysis • Scripting • Data Import/Export

“Transforming Data into Intelligent Decisions”

KASHIF MAQBOOL JOIYA

Data Scientist | AI Engineer

GitHub

https://github.com/KashifMaqbool

Table of Contents

Module 1

1 Introduction to Python

1.1 Users of Python

Python is widely used by professionals across different fields because of its clear and readable syntax.

1.1.1 Experienced Programmers

- Can develop the same programs from other languages with less code.

1.1.2 Beginners

- Python is an excellent starting language due to its large global community and wealth of documentation.

1.2 Benefits of Using Python

Python is a powerful, high-level, general-purpose programming language. It offers:

- A large standard library with tools for:
  - Databases
  - Automation
  - Web scraping
  - Text processing
  - Image processing
  - Machine learning
  - Data analytics
- Applications across:
  - **Data Science**
  - **Artificial Intelligence and Machine Learning**
  - **Web Development**
  - **Internet of Things (IoT)**
- A strong global community and support from the Python Software Foundation.

1.3 Popularity of Python

Python has become the most widely used and popular programming language in the data science industry:

- **2019 Kaggle Survey**: 75% of over 10,000 respondents reported using Python regularly.
- **Glassdoor (2019)**: More than 75% of data science job listings included Python.
- Over 80% of data professionals worldwide reported using Python in 2019 surveys.

1.3.1 Organizations Using Python

Major organizations that rely on Python include:

- IBM
- Wikipedia
- Google
- Yahoo!
- CERN
- NASA
- Facebook
- Amazon
- Instagram
- Spotify
- Reddit

1.4 Python in Data Science and AI

Python's rich ecosystem of libraries supports advanced applications:

1.4.1 Scientific Computing Libraries

- Pandas
- NumPy
- SciPy
- Matplotlib

1.4.2 Artificial Intelligence and Machine Learning

- TensorFlow
- PyTorch
- Keras
- Scikit-learn

1.4.3 Natural Language Processing

- Natural Language Toolkit (NLTK)

1.5 Diversity and Inclusion in the Python Community

Python's community is known for its commitment to diversity and inclusion:

- Python Software Foundation
- PyLadies

2 Course Overview

Welcome to the **Python for Data Science, AI, and Development**. After completing this, you will:

- Possess basic knowledge of Python.
- Understand different data types.
- Use lists, tuples, dictionaries, and sets.
- Apply conditions and branching.
- Implement loops and create functions.
- Perform exception handling.
- Create and use objects.
- Read and write files.
- Collect data using APIs and web scraping.

2.1 Course Content

This course is divided into five modules. Aim to complete at least one module per week.

2.1.1 Module 1: Python Basics

- About the Course
- Types
- Expressions and Variables
- String Operations

2.1.2 Module 2: Python Data Structures

- Lists and Tuples
- Dictionaries
- Sets

2.1.3 Module 3: Python Programming Fundamentals

- Conditions and Branching
- Loops
- Functions
- Exception Handling
- Objects and Classes
- Practice with Python Programming Fundamentals

2.1.4 Module 4: Working with Data in Python

- Reading and Writing Files with Open
- Pandas
- NumPy in Python

2.1.5 Module 5: APIs and Data Collection

- Simple APIs
- REST APIs, Web Scraping, and Working with Files
- Final Exam

3 Getting Started with Jupyter

3.1 Introduction

Welcome to **Getting Started with Jupyter**. After completing this module, you will be able to:

- Describe how to run, insert, and delete a cell in a notebook.
- Work with multiple notebooks.
- Present a notebook.
- Shut down a notebook session.

3.2 Launching a Notebook

In the lab session of this module, you can launch a notebook using the Skills Network virtual environment.

- Select the check box, click **Open Tool**, and the environment will launch Jupyter Lab.
- Once the notebook opens, you can rename it by clicking **File > Rename Notebook** and entering a new name.

3.2.1 Running Your First Cell

- Create a new notebook and type `print("Hello World")`.
- Click the **Run** button to execute the cell.
- Alternatively:
  - From the main menu bar, click **Run > Run Selected Cells**.
  - Or press **Shift + Enter** as a shortcut.
- To run all cells in the notebook, select **Run All Cells**.

3.3 Working with Cells

3.3.1 Inserting a Cell

- Click the **plus (+)** symbol in the toolbar to insert a new cell.

3.3.2 Deleting a Cell

- Highlight the cell, then click **Edit > Delete Cells**.
- Shortcut: Press **D** twice (**DD**) on the highlighted cell.

3.3.3 Moving Cells

- You can move cells up or down as needed.

3.4 Working with Multiple Notebooks

- Click the **plus (+)** button in the toolbar and select the file you want to open.
- Alternatively: **File > Open New Launcher** or **File > Open New Notebook**.
- Opened notebooks can be arranged side by side.

3.4.1 Example

In one notebook, assign:

```
one = 1
two = 2
print(one + two)
```

This demonstrates working with multiple notebooks simultaneously.

3.5 Presenting with Jupyter

Jupyter supports creating presentations directly from notebooks:

- Use **Markdown cells** to add titles, text, and descriptions.
- Create plots and combine them with code outputs.
- Convert cells into slides or sub-slides for presentations.

This feature allows seamless delivery of code, visualizations, and narrative as part of a project.

3.6 Shutting Down Notebooks

When you finish working:

- Click the **stop icon** on the sidebar (second icon from the top).
- You can terminate all sessions at once or shut them down individually.
- Once shut down, you will see **"No Kernel"** in the top-right corner, confirming the notebook is inactive.
- Close the tabs after shutting down.

3.7 Summary

In this module, you learned how to:

- Run, insert, and delete code cells.
- Work with multiple notebooks at the same time.
- Present results using Markdown and code cells.
- Shut down notebook sessions when finished.

4 Module 1 Summary: Python Basics

4.1 Data Types in Python

Python can distinguish among various data types:

- **Integers**: Whole numbers that can be positive or negative.
- **Floats**: Numbers with decimal points that represent whole or fractional values.
- **Strings**: Text data enclosed in single or double quotes, consisting of letters, digits, whitespace, or special characters.
- **Booleans**: Represent logical values, `True` or `False`.

4.1.1 Typecasting

- Integers can be converted to floats and vice versa.
- Integers and floats can be converted to strings.
- Integers or floats can be converted to Booleans:
  - 0 → `False`
  - Non-zero values → `True`

4.2 Expressions and Operations

Expressions in Python combine values and operations to produce results.

- Supports mathematical operations such as addition, subtraction, multiplication, and division.
- // performs integer division, discarding the fractional part.
- Python follows the **BODMAS** order of operations when evaluating expressions.

4.3 Variables

Variables store and manipulate data in Python:

- The assignment operator (=) assigns values to variables.
- Reassigning a variable overrides its previous value.
- Mathematical operations can be performed on variables.
- Changing one variable affects others only if they reference the same **mutable object**.

4.4 String Operations

Python provides powerful operations for manipulating strings:

- **String Basics**:
  - Strings are ordered sequences of characters.
  - Characters are indexed using positive and negative indices.
  - Strings support indexing, slicing, concatenation, and replication.
  - Strings are **immutable** and cannot be changed once created.
- **Escape Sequences**:

    - \n → New line
    - \t → Tab
    - \\ → Backslash
- **String Methods**:
    - Search, modify, and format strings.
    - Change case, replace characters, and find items.
    - Applying a method to a string creates and returns a new string.

4.5 Key Takeaways

- Python supports multiple data types: integers, floats, strings, and Booleans.
- Typecasting allows conversion between data types.
- Expressions and operators enable mathematical calculations.
- Variables help store and manipulate data.
- Strings offer rich operations like slicing, formatting, and built-in methods, but remain immutable.

Module 2

5 Lists and Tuples

5.1 Compound Data Types

Lists and tuples are examples of compound data types in Python. They are key data structures used to organize and manage collections of data.

5.2 Tuples

5.2.1 Definition

- Tuples are **ordered sequences**.
- Represented as comma-separated elements enclosed in parentheses ( ).
- They can contain multiple data types (strings, integers, floats, etc.), but the variable type remains a **tuple**.

5.2.2 Accessing Elements

- Elements are accessed using **indexing** with square brackets [].
- Indexing starts from 0.
- Negative indexes can be used to access elements from the end of the tuple.

5.2.3 Operations on Tuples

- **Concatenation**: Tuples can be combined using the + operator.
- **Slicing**: A range of elements can be accessed using [start:end]. The end index is exclusive.
- **Length**: The len() function returns the number of elements in a tuple.

5.2.4 Immutability

- Tuples are **immutable** (cannot be modified after creation).
- Assigning one tuple variable to another references the same object.
- To modify a tuple, a **new tuple** must be created.

5.2.5 Functions with Tuples

- The sorted() function creates a **new sorted list** from a tuple.
- Tuples can also contain other tuples or complex objects, known as **nesting**.

5.2.6 Nesting and Indexing

- Tuples can be nested (tuples within tuples).
- Standard indexing rules apply to access nested elements.
- This nesting can be visualized like a **tree structure**.
- Example: tup[2][3] -> means the fourth element of the sub-tuple at index 2 inside a tuple.

5.3 Lists

5.3.1 Definition

- Lists are **ordered sequences** represented with square brackets [ ].
- Lists are **mutable**, meaning their contents can be changed.

5.3.2 Accessing Elements

- Elements are accessed using indexing, similar to tuples.
- Negative indexes are supported.
- Slicing works the same as in tuples.

5.3.3 Operations on Lists

- **Concatenation**: Lists can be combined using the + operator.
- **Extend**: The .extend() method adds multiple elements to the end of a list.
- **Append**: The .append() method adds a single element.
- **Deletion**: Elements can be removed using the del command.

5.3.4 Mutability

- Lists can be modified directly:
  - Changing an element by index.
  - Adding new elements using methods.
  - Deleting elements.
- **Aliasing**: Assigning one list variable to another references the same object. Changes in one affect the other. e.g. A = [1,2,3] and B = A.
- **Cloning**: To avoid aliasing, lists can be cloned so each variable references a separate object.

5.3.5 Conversion

- Strings can be converted into lists using the .split() method.
- A delimiter can be passed to .split() to control how the string is divided.

5.4 Summary

- **Tuples**: Ordered, immutable, represented by parentheses ( ).
- **Lists**: Ordered, mutable, represented by square brackets [ ].
- Both support indexing, slicing, concatenation, and nesting.
- Tuples are often used when data should remain constant, while lists are preferred when modifications are needed.

6 Dictionaries

6.1 Definition

- Dictionaries are **collections of key-value pairs** in Python.
- Represented using curly brackets { }.
- Keys act like indexes but do not have to be integers. They are usually strings and must be **immutable** and **unique**.
- Values can be immutable, mutable, and can include duplicates.

6.2 Structure

- Each **key** is followed by a **value**, separated by a colon :.
- Each key-value pair is separated by a comma.

6.3 Example

```
albums = {
   "Back in Black": 1980,
   "The Dark Side of the Moon": 1973,
```

```
    "The Bodyguard": 1992,
    "Thriller": 1982
}
```

- Keys: Album titles (e.g., "Back in Black").
- Values: Release years (e.g., 1980).

6.4 Accessing Values

- Values are accessed using **square brackets** with the key.

```
albums["Back in Black"]  # Output: 1980
albums["The Dark Side of the Moon"]  # Output: 1973
```

6.5 Modifying Dictionaries

- **Add Entry**: Assign a new key with a value.

```
albums["Graduation"] = 2007
```

- **Delete Entry**: Use the `del` command.

```
del albums["Thriller"]
```

6.6 Checking Membership

- Use the `in` command to check if a key exists.

```
"Back in Black" in albums   # Output: True
"Random Album" in albums    # Output: False
```

6.7 Dictionary Methods

- **Keys**: `albums.keys()` returns all keys.
- **Values**: `albums.values()` returns all values.

6.8 Visualization

- Think of a dictionary like a **table**:
    - The first column represents **keys** (e.g., album titles).
    - The second column represents **values** (e.g., release years).

6.9 Summary

- Dictionaries store data as key-value pairs.
- Keys must be unique and immutable.
- Values can be duplicates and may be mutable or immutable.
- They are powerful for lookups, modifications, and managing structured data.

7 Sets

7.1 Definition

- Sets are a **collection type** in Python.
- Like lists and tuples, they can contain different data types.
- **Key Characteristics**:
    - **Unordered**: They do not record element positions.
    - **Unique Elements**: Duplicate values are not allowed.

7.2 Creating Sets

- Defined using **curly brackets** `{}`.
- Duplicate items are automatically removed.
- You can convert a list to a set using the `set()` function (**typecasting**).

```
my_list = ["ACDC", "BackInBlack", "ACDC"]
my_set = set(my_list)
print(my_set)  # Output: {"ACDC", "BackInBlack"}
```

7.3 Basic Set Operations

7.3.1 Adding Elements

- Use the `add()` method.

```
A = {"Thriller", "ACDC"}
A.add("inSync")
```

- Adding the same item again has no effect (no duplicates).

7.3.2 Removing Elements

- Use the `remove()` method.

```
A.remove("inSync")
```

7.3.3 Membership Test

- Use the `in` keyword.

```
"ACDC" in A   # Output: True
"Who" in A    # Output: False
```

7.4 Mathematical Set Operations

7.4.1 Intersection

- The intersection of two sets contains only the elements present in **both sets**.

```
AlbumSet1 = {"ACDC", "BackInBlack", "Thriller"}
AlbumSet2 = {"ACDC", "BackInBlack", "The Bodyguard"}
AlbumSet3 = AlbumSet1 & AlbumSet2
print(AlbumSet3)  # Output: {"ACDC", "BackInBlack"}
```

7.4.2 Union

- The union of two sets contains **all unique elements** from both sets.

```
AlbumSet1 | AlbumSet2
```

7.4.3 Subset

- Check if a set is a subset of another using issubset().

```
AlbumSet3.issubset(AlbumSet1)  # Output: True
```

7.5 Visualization with Venn Diagrams

- **Sets** can be represented as circles.
- **Intersection**: Overlapping area.
- **Union**: A Combination of both circles.
- **Subset**: One circle fully inside another.

7.6 Summary

- Sets are unordered collections of unique elements.
- They support operations like **add**, **remove**, and **membership checks**.
- Mathematical operations include **intersection**, **union**, and **subset checking**.
- Useful for managing collections with no duplicates and for mathematical computations.

8 Module 2 Summary: Python Data Structures

8.1 Tuples

- Ordered and immutable collections of elements.
- Defined using **parentheses ()** with comma-separated values.
- Can include strings, integers, floats, and even nested tuples.
- Access elements using positive and negative indexing.
- Operations include combining, concatenating, and slicing.
- Since tuples are immutable, new tuples must be created for modifications.

8.2 Lists

- Ordered and mutable collections of items.
- Defined using **square brackets []**.
- Can contain mixed data types such as strings, integers, floats, and nested lists.
- Elements are accessed using positive and negative indexing.
- Support operations like adding, deleting, splitting, concatenating, and appending.
- Aliasing occurs when multiple names refer to the same list object.
- Lists can be cloned to create independent copies.

8.3 Dictionaries

- Store **key-value pairs** for flexible data retrieval.

- Defined using **curly brackets** {}.
- Keys must be immutable and unique, while values can be mutable and allow duplicates.
- Key-value pairs are separated by commas.
- Access values using keys.
- Support operations like adding, deleting, and checking for the existence of keys (returns True/False).
- Provide methods to retrieve lists of keys and values.

8.4 Sets

- Unordered collections of **unique elements**.
- Defined using **curly brackets** {}.
- Automatically remove duplicate items.
- A list passed through the set() function generates a set of unique elements.
- Support set operations such as **adding, removing, and membership checks**.
- Mathematical operations include:
    - **Intersection (&)** → Common elements between sets.
    - **Union (|)** → All unique elements from both sets.
    - **Subset check (issubset())** → Determine if one set is contained within another.

8.5 Summary

- **Tuples**: Ordered, immutable, suitable for fixed collections.
- **Lists**: Ordered, mutable, versatile for dynamic collections.
- **Dictionaries**: Key-value storage with unique keys and flexible values.
- **Sets**: Unordered, unique elements, ideal for duplicate removal and set operations.

Module 3

9 Exploring Python Functions

9.1 Objectives

After reading this, you should be able to:

- Describe the function concept and the importance of functions in programming
- Write a function that takes inputs and performs tasks
- Use built-in functions like len(), sum(), and others effectively
- Define and use your own functions in Python
- Differentiate between global and local variable scopes
- Use loops within a function
- Modify data structures using functions

9.2 Introduction to Functions

A function is a fundamental building block that encapsulates specific actions or computations. Similar to mathematics, functions in programming take inputs, perform operations, and return outputs.

9.3 Purpose of Functions

Functions promote **modularity** and **reusability**. Instead of duplicating code, you can define a function once and call it wherever needed, making code cleaner and easier to maintain.

9.4 Benefits of Using Functions

- **Modularity**: Break complex tasks into manageable parts
- **Reusability**: Use functions multiple times without rewriting code
- **Readability**: Enhance understanding with meaningful function names
- **Debugging**: Isolated functions simplify troubleshooting
- **Abstraction**: Simplify complex processes with easy-to-use interfaces
- **Collaboration**: Teams can work on separate functions concurrently
- **Maintenance**: Changes in a function reflect everywhere it is used

9.5 How Functions Work

9.5.1 Inputs (Parameters)

Functions receive inputs (parameters) that allow them to work with specific data.

9.5.2 Performing Tasks

Functions execute predefined tasks such as calculations, operations on data, formatting, or fetching information.

9.5.3 Producing Outputs

Functions return results (outputs) that can be reused, stored in variables, or passed to other functions.

Example:

```
def calculate_total(a, b):  # Parameters: a and b
    total = a + b           # Task: Addition
    return total            # Output: Sum

result = calculate_total(5, 7)
print(result)  # Output: 12
```

9.6 Python's Built-in Functions

Python offers many built-in functions that simplify coding tasks.

9.6.1 Common Examples

```
# len()
string_length = len("Hello, World!")  # Output: 13
list_length = len([1, 2, 3, 4, 5])    # Output: 5

# sum()
total = sum([10, 20, 30, 40, 50])     # Output: 150
```

```
# max()
highest = max([5, 12, 8, 23, 16])     # Output: 23

# min()
lowest = min([5, 12, 8, 23, 16])      # Output: 5
```

9.7 Defining Your Own Functions

```
def function_name():
    pass
```

- `pass` is a placeholder statement to maintain correct syntax until code is added.

9.7.1 Example with Parameters

```
def greet(name):
    return "Hello, " + name

print(greet("Alice"))  # Output: Hello, Alice
```

9.7.2 Docstrings

```
def multiply(a, b):
    """
    This function multiplies two numbers.
    Input: a (number), b (number)
    Output: Product of a and b
    """
    print(a * b)

multiply(2, 6)  # Output: 12
```

9.7.3 Return Statement

```
def add(a, b):
    return a + b

sum_result = add(3, 5)
print(sum_result)  # Output: 8
```

9.8 Understanding Scopes and Variables

- **Global Scope**: Variables defined outside functions (accessible everywhere)
- **Local Scope**: Variables defined inside a function (accessible only inside it)

```
global_variable = "I'm global"

def example_function():
    local_variable = "I'm local"
    print(global_variable)  # Access global
    print(local_variable)   # Access local

example_function()
print(global_variable)      # Works
# print(local_variable)     # Error
```

9.9 Using Functions with Loops

Functions can contain loops, making tasks more organized and reusable.

```
def print_numbers(limit):
    for i in range(1, limit + 1):
        print(i)
```

```
print_numbers(5)  # Output: 1 2 3 4 5
```

9.9.1 Example with Loop and Function

```
def greet(name):
    return "Hello, " + name

for _ in range(3):
    print(greet("Alice"))
```

9.10 Modifying Data Structures with Functions

9.10.1 Adding and Removing Elements in a List

```
# Initial list
my_list = []

# Function to add elements
def add_element(data_structure, element):
    data_structure.append(element)

# Function to remove elements
def remove_element(data_structure, element):
    if element in data_structure:
        data_structure.remove(element)
    else:
        print(f"{element} not found in the list.")

# Add elements
add_element(my_list, 42)
add_element(my_list, 17)
add_element(my_list, 99)

print("Current list:", my_list)

# Remove elements
remove_element(my_list, 17)
remove_element(my_list, 55)  # Not found

print("Updated list:", my_list)
```

10 Exception Handling in Python

10.1 Objectives

By the end of this reading, you should be able to:

- Understand exceptions
- Distinguish errors from exceptions
- Recognize common Python exceptions
- Manage exceptions effectively

10.2 What Are Exceptions?

Exceptions are alerts that occur when something unexpected happens while running a program. They could result from coding mistakes or unplanned situations. Python can raise exceptions automatically, but we can also trigger them manually using the `raise` statement. By handling exceptions, we can prevent programs from crashing.

10.3 Errors vs. Exceptions

Errors and exceptions differ in their severity and handling:

Aspect	Errors	Exceptions
Origin	Caused by environment, hardware, or operating system	Caused by problematic code execution within the program
Nature	Severe, often causing program crashes	Less severe, can usually be fixed so the program continues
Handling	Not usually caught or handled by the program	Can be caught using try-except blocks
Examples	SyntaxError (incorrect syntax), NameError (undefined variable)	ZeroDivisionError, FileNotFoundError
Categorization	Not classified	Categorized into classes (ArithmeticError, IOError, ValueError, etc.)

10.4 Common Exceptions in Python

Here are some commonly encountered exceptions:

10.4.1 ZeroDivisionError

Occurs when dividing a number by zero.

```
result = 10 / 0  # Raises ZeroDivisionError
```

10.4.2 ValueError

Occurs when inappropriate values are used, e.g., converting a non-numeric string to an integer.

```
num = int("abc")  # Raises ValueError
```

10.4.3 FileNotFoundError

Occurs when trying to access a non-existent file.

```
with open("nonexistent_file.txt", "r") as file:
    content = file.read()  # Raises FileNotFoundError
```

10.4.4 IndexError

Occurs when accessing a list index out of range.

```
my_list = [1, 2, 3]
missing = my_list[5]  # Raises IndexError
```

10.4.5 KeyError

Occurs when accessing a non-existent dictionary key.

```
my_dict = {"name": "Alice", "age": 30}
missing = my_dict["city"]  # Raises KeyError
```

10.4.6 TypeError

Occurs when using an object in an incompatible manner.

```
result = "hello" + 5  # Raises TypeError
```

10.4.7 AttributeError

Occurs when accessing an attribute or method that does not exist.

```
text = "example"
missing = text.some_method()  # Raises AttributeError
```

10.4.8 ImportError

Occurs when attempting to import a non-existent module.

```
import non_existent_module  # Raises ImportError
```

Note: These are just a few examples; Python has many exceptions. With proper handling, you can manage them effectively.

10.5 Handling Exceptions

Python uses **try-except** blocks to handle exceptions and prevent crashes.

10.5.1 How It Works

- Code that might raise an exception is placed in the `try` block.
- If an exception occurs, control jumps to the `except` block.
- The `except` block defines how to handle the exception gracefully.
- The program continues execution after handling the exception.

10.5.2 Example: Division by Zero

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Error: Cannot divide by zero")

print("Outside of try and except block")
```

11 Objects and Classes in Python

Estimated time needed: 20 minutes

11.1 Objectives

By the end of this reading, you should be able to:

- Understand objects and classes in Python
- Identify data attributes and methods
- Create your own classes and objects

- Work with constructors and methods
- Recognize how to modify and interact with objects

11.2 Introduction to Objects

Python has many different data types: integers, floats, strings, lists, dictionaries, and Booleans. In Python, each is an **object**.

11.2.1 Characteristics of an Object

Every object has:

- A **type**
- An **internal representation**
- A set of **functions (methods)** to interact with the data

An **object** is an instance of a particular type. For example:

- Each integer you create is an integer object.
- Each list you create is a list object.

We can find out the type of an object by using the `type()` command.

11.2.2 Examples of Objects

- Integer object
- List object
- String object
- Dictionary object

11.3 Classes and Methods

A **class** is a blueprint for creating objects. It defines:

- **Data attributes/State**: characteristics of the object
- **Methods/Behavior**: functions that interact with the object

We have already used methods, such as `list.sort()` or `list.reverse()`, which change the state of the list object.

11.4 Creating Your Own Classes

You can create your own classes in Python. A class defines the structure of objects:

11.4.1 Circle Class Example

Data attributes:

- radius
- color

11.4.2 Rectangle Class Example

Data attributes:

- color
- height
- width

11.4.3 Class Definition

To create a class in Python:

```
class Circle(object):
    def __init__(self, radius, color):
        self.radius = radius
        self.color = color
```

11.4.4 Creating Objects from a Class

```
circle1 = Circle(4, "red")
circle2 = Circle(2, "green")
```

Similarly, you can create rectangle objects with height, width, and color.

11.5 Constructors and the __init__ Method

The __init__ method is a **constructor**. It initializes the object when it is created.

- self refers to the instance of the class.
- Parameters (e.g., radius, color) initialize the object's data attributes.

```
class Rectangle(object):
    def __init__(self, height, width, color):
        self.height = height
        self.width = width
        self.color = color
```

11.6 Accessing and Modifying Attributes

You can access attributes with the dot operator:

```
print(circle1.radius)
print(circle1.color)
```

You can also modify them directly:

```
circle1.color = "blue"
```

11.7 Methods in Classes

Methods are functions inside a class that interact with its data attributes.

11.7.1 Example: Adding to Radius

```
class Circle(object):
    def __init__(self, radius, color):
        self.radius = radius
        self.color = color

    def add_radius(self, r):
        self.radius = self.radius + r
```

11.7.2 Using the Method

```
circle = Circle(2, "red")
circle.add_radius(8)
print(circle.radius)  # Output: 10
```

11.8 Drawing Methods

You can also define methods like drawCircle or drawRectangle to visualize objects. (See lab exercises for implementation.)

11.9 Summary

- Objects are **instances of classes**.
- Classes define **data attributes** and **methods**.
- The __init__ method is the constructor used to initialize object attributes.
- Methods allow interaction with and modification of object data.
- You can use the dir() function to inspect available attributes and methods of an object.

For more advanced details, visit python.org.

12 Module 3 Summary: Python Programming Fundamentals

12.1 Conditional Statements

- Python conditions use **if statements** to execute code based on true/false conditions created by comparisons and Boolean expressions.
- **Comparison operators** include:
    - == (equal to)
    - > (greater than)
    - < (less than)
    - != (not equal to)
- You can compare **integers, strings, and floats**.
- Python branching uses if, else, and elif to direct program flow and execute different code blocks.
- if defines actions when the condition is true.
- else defines actions when all previous conditions are false.
- elif provides additional checks only if the initial condition is false.
- Boolean logic operators are used to perform operations on Boolean values.

12.2 Loops

- Loops are control structures that automate repetitive tasks and iterate over data structures.
- The range() function generates a sequence of numbers with a start, stop, and step value.
- **For loop** iterates over sequences like lists, tuples, or strings, executing code for each item.
- **While loop** executes a block of code as long as the condition remains true.

12.3 Functions

- Functions are reusable code blocks that perform tasks, accept inputs, and often return results.
- Python has many **built-in functions**, such as:
  - len() to find the length of a sequence
  - sum() to calculate the sum of a sequence
  - sorted() to return a sorted list
  - sort() to sort items in the original list
- You can also **create your own functions**.
- Functions should include **documentation strings** (docstrings) to ensure clarity and maintainability.
- The help() command displays documentation for a function.
- Functions can have multiple parameters.
- If no return statement is used, the function returns **None** by default.
- The pass keyword can be used as a placeholder.
- Functions usually perform multiple tasks.

12.4 Variable Scope

- The scope of a variable defines where it can be accessed or modified.
- **Local scope**: variables defined inside a block or function, accessible only within it.
- **Global scope**: variables defined at the top level, accessible throughout the program.

12.5 Exception Handling

- Exception handling prevents errors from crashing a program.
- try-except is used to handle errors.
- try-except-else adds an else block that runs when no exceptions occur.
- try-except-else-finally ensures the finally block always runs, regardless of exceptions.

12.6 Objects and Classes

- Objects are **instances of classes** that encapsulate data and behavior.
- The type() function determines the type of an object.
- **Classes** are blueprints for creating objects, defining attributes and methods.
- **Methods** can modify an object's state while maintaining its type.
- The __init__ method initializes object attributes.
- Instances of a class can be created with specific attributes.

- **Data attributes** define the data of an object.
- **Methods** are functions that interact with and modify attributes.
- Methods require self as the first parameter, along with other parameters.

Module 4

13 Reading a File with Open()

Estimated Time Needed: 10 minutes

13.1 Introduction

File handling is an essential aspect of programming, and Python provides built-in functions to interact with files. This guide explores how to use Python's open() function to read text files (.txt files).

13.2 Objectives

- Describe how to use the open() and read() Python functions to open and read the contents of a text file.
- Explain how to use the with statement in Python.
- Describe how to use the readline() function in Python.
- Explain how to use the seek() function to read specific character(s) in a text file.

13.3 Plain Text Files

Plain text files contain unformatted text without any specific structure. You can read these files line by line or load all the content into memory.

13.4 Opening the File

There are two primary methods to open a file in Python.

13.4.1 1. Using Python's open() Function

Suppose we have a file named file.txt. The open() function creates a file object and allows access to the file's contents. It takes two key parameters:

- **File Path:** The file name and its directory.
- **Mode:** Specifies the purpose of opening the file (e.g., 'r' for reading, 'w' for writing, 'a' for appending).

```
# Open the file in read ('r') mode
file = open('file.txt', 'r')
```

This line opens file.txt in read mode and returns a file object stored in the variable file.

13.4.2 2. Using the with Statement

The with statement simplifies file handling by automatically closing the file when operations within its block are completed.

```
# Open the file using 'with' in read ('r') mode
with open('file.txt', 'r') as file:
    # further code
```

The with statement ensures the file is properly closed after operations, even if an exception occurs.

13.4.2.1 Advantages of Using the with Statement

- **Automatic Resource Management:** The file closes automatically when exiting the with block.
- **Cleaner Code:** No need to explicitly call close(), making code concise and less error-prone.

Note: For most file reading and writing operations, using the with statement is best practice.

13.5 Reading Operations

13.5.1 1. Reading the Entire Content

You can read the entire content of a file using the read() method. The data is stored as a string in a variable.

```
# Reading and Storing the Entire Content of a File
with open('file.txt', 'r') as file:
    file_stuff = file.read()
    print(file_stuff)
```

Step-by-Step Explanation:

1. **Open the File:** Open file.txt in read mode using with.
2. **Read Content:** Use read() to get the entire file content.
3. **Process Content:** The file's data is now in file_stuff; you can display or manipulate it.
4. **Automatic Closure:** The file automatically closes after the block ends.

13.5.2 2. Reading the Content Line by Line

Python provides multiple methods for reading files line by line.

13.5.2.1 Using readlines()

Reads the entire file and stores each line as an element in a list.

13.5.2.2 Using readline()

Reads one line at a time and can be called repeatedly.

```
file = open('file.txt', 'r')
line1 = file.readline()
line2 = file.readline()

print(line1)
if 'important' in line2:
   print('This line is important!')

while True:
   line = file.readline()
   if not line:
      break
   print(line)

file.close()
```

Explanation:

- Each call to readline() reads the next line.
- The loop continues until there are no more lines.
- Always close the file using close() when finished.

13.5.3 3. Reading Specific Characters

You can specify how many characters to read using the read() method.

13.5.3.1 Steps:

1. **Open the File:**

```
file = open('file.txt', 'r')
```

2. **Navigate to a Position (Optional):**
   Use seek() to move the file pointer to a specific position.

```
file.seek(10)  # Moves to the 11th byte
```

3. **Read Characters:**

```
characters = file.read(5)  # Reads 5 characters
print(characters)
```

4. **Close the File:**

```
file.close()
```

13.6 Conclusion

In conclusion, file handling is a fundamental aspect of programming. Python's built-in functions like open(), read(), readline(), and seek() make file operations simple and efficient. The use of the with statement ensures proper resource management, making it the recommended approach for file operations.

14 Writing on a File with Open()

14.1 Objective

- Create and write data to a file in Python.
- Write multiple lines of text to a file using lists and loops.
- Add new information to an existing file without erasing its content.
- Compare and contrast different file modes in Python, understanding their meanings and use cases.

14.2 Writing to a File

You can create a new text file and write data to it using Python's open() function. The open() function takes two primary arguments:

- **File path:** The name and directory of the file.
- **Mode:** Specifies the operation you want to perform on the file.

For writing, use the mode 'w'. Here's an example:

```
# Create a new file Example2.txt for writing
with open('Example2.txt', 'w') as file1:
    file1.write("This is line A\n")
    file1.write("This is line B\n")
    # file1 is automatically closed when the 'with' block exits
```

14.2.1 Code Explanation

- **Line 2:** The open() function opens or creates a file named Example2.txt for writing ('w' mode). The with statement ensures the file is automatically closed when the block exits.
- **Line 3:** The write() method writes the text *This is line A* followed by a newline (\n).
- **Line 4:** Writes *This is line B* to a new line in the same file.

14.3 Writing Multiple Lines to a File Using a List and Loop

You can store multiple lines of text in a list and write them to a file using a loop.

```
# List of lines to write to the file
Lines = ["This is line 1", "This is line 2", "This is line 3"]

# Create a new file Example3.txt for writing
with open('Example3.txt', 'w') as file2:
    for line in Lines:
        file2.write(line + "\n")
```

```
    # file2 is automatically closed when the 'with' block exits
```

14.3.1 Code Explanation

- **Line 2:** A list named Lines stores multiple strings to be written to the file.
- **Line 5:** Opens Example3.txt for writing using 'w' mode.
- **Line 6–7:** Iterates over each line in the list and writes it to the file, appending a newline character.
- **Line 8:** The file closes automatically at the end of the with block.

14.4 Appending Data to an Existing File

You can use the 'a' mode to append new data to an existing file without overwriting its contents.

```
# Data to append to the existing file
new_data = "This is line C"

# Open an existing file Example2.txt for appending
with open('Example2.txt', 'a') as file1:
    file1.write(new_data + "\n")
    # file1 is automatically closed when the 'with' block exits
```

14.4.1 Code Explanation

- **Line 2:** Defines the variable new_data containing the text to append.
- **Line 5:** Opens Example2.txt in 'a' mode. If the file doesn't exist, it is created.
- **Line 6:** Appends the new data to the file, adding a newline at the end.
- **Line 7:** The file automatically closes after exiting the with block.

14.5 Copying Contents from One File to Another

You can copy the contents of one file to another by reading from the source file and writing to the destination file.

```
# Open the source file for reading
with open('source.txt', 'r') as source_file:
    # Open the destination file for writing
    with open('destination.txt', 'w') as destination_file:
        # Read lines from the source file and copy them to the destination file
        for line in source_file:
            destination_file.write(line)
    # Destination file is automatically closed when the 'with' block exits
# Source file is automatically closed when the 'with' block exits
```

14.5.1 Code Explanation

- **Line 2:** Opens source.txt in read ('r') mode.
- **Line 4:** Opens destination.txt in write ('w') mode.
- **Line 6–7:** Loops through each line of the source file and writes it to the destination file.
- **Line 8–9:** Both files close automatically when their respective with blocks exit.

14.6 File Modes in Python

The following table lists different file modes, their syntax, and common use cases:

Mode	Syntax	Description
Read	'r'	Opens an existing file for reading. Raises an error if the file doesn’t exist.
Write	'w'	Creates a new file for writing. Overwrites the file if it already exists.
Append	'a'	Opens a file for appending data. Creates the file if it doesn’t exist.
Exclusive	'x'	Creates a new file for writing. Raises an error if the file already exists.
Read Binary	'rb'	Opens a binary file for reading.
Write Binary	'wb'	Creates a binary file for writing.
Append Binary	'ab'	Opens a binary file for appending data.
Exclusive Binary	'xb'	Creates a binary file for writing. Raises an error if it already exists.
Read Text	'rt'	Opens a text file for reading (default).
Write Text	'wt'	Creates a new text file for writing.
Append Text	'at'	Opens a text file for appending data.
Exclusive Text	'xt'	Creates a text file for writing but raises an error if it already exists.
Read & Write	'r+'	Opens an existing file for both reading and writing.
Write & Read	'w+'	Creates a new file for reading and writing. Overwrites if it already exists.
Append & Read	'a+'	Opens a file for both appending and reading. Creates the file if it doesn’t exist.
Exclusive Read & Write	'x+'	Creates a new file for reading and writing. Raises an error if it already exists.

14.7 Conclusion

Working with files is a fundamental part of programming. Python provides flexible tools like open(), write(), and with statements to create, write, append, and copy files efficiently. Understanding file modes ensures proper use of resources and data management in any Python project.

15 Introduction to Pandas for Data Analysis

15.1 Objectives

- Learn what Pandas Series are and how to create them.
- Understand how to access and manipulate data within a Series.
- Discover the basics of creating and working with Pandas DataFrames.
- Learn how to access, modify, and analyze data in DataFrames.
- Gain insights into common DataFrame attributes and methods.

15.2 What is Pandas?

Pandas is a popular open-source data manipulation and analysis library for Python. It provides a powerful and flexible set of tools for working with structured data, making it a fundamental tool for data scientists, analysts, and engineers.

Pandas can handle data in various formats, such as tabular and time-series data, making it an essential part of the data processing workflow in many industries.

15.2.1 Key Features of Pandas

- **Data Structures:** Pandas offers two main data structures:
  - **DataFrame:** A two-dimensional, size-mutable table with labeled axes (rows and columns).
  - **Series:** A one-dimensional labeled array, representing a single column or row of data.
- **Data Import and Export:** Easily read and write data from sources like CSV, Excel, SQL, and more.
- **Data Merging and Joining:** Combine multiple DataFrames using methods like merge and join, similar to SQL operations.
- **Efficient Indexing:** Quickly access specific rows and columns using indexing and selection methods.
- **Custom Data Structures:** Create and manipulate data in customized ways to fit specific needs.

15.3 Importing Pandas

To use Pandas, you must first import it into your Python environment. It is commonly imported using the alias pd:

```
import pandas as pd
```

15.4 Data Loading

Pandas allows you to load data from various sources, such as CSV and Excel files. Use the read_csv() function to load CSV data into a DataFrame:

```
import pandas as pd
# Read the CSV file into a DataFrame
df = pd.read_csv('your_file.csv')
```

Replace 'your_file.csv' with the actual file path. Ensure the file is located in your working directory or provide the correct path.

15.5 What is a Series?

A **Series** is a one-dimensional labeled array in Pandas. It can be thought of as a single column of data with labels (indices) for each element. A Series can be created from lists, NumPy arrays, or dictionaries.

```
import pandas as pd
# Create a Series from a list
data = [10, 20, 30, 40, 50]
s = pd.Series(data)
print(s)
```

Pandas automatically assigns numerical indices (0, 1, 2, 3, 4), but you can specify custom labels if needed.

15.5.1 Accessing Elements in a Series

You can access Series elements using index labels or integer positions.

```
print(s[2])     # Access by label (value 30)
print(s.iloc[3]) # Access by position (value 40)
print(s[1:4])   # Access a range of elements
```

15.5.2 Series Attributes and Methods

Some useful attributes and methods of a Series include:

- values – Returns data as a NumPy array.
- index – Returns index labels.
- shape, size – Returns dimensions and size.
- mean(), sum(), min(), max() – Summary statistics.
- unique(), nunique() – Unique values and their count.
- sort_values(), sort_index() – Sort by values or index.
- isnull(), notnull() – Detect missing values.
- apply() – Apply custom functions to elements.

15.6 What is a DataFrame?

A **DataFrame** is a two-dimensional labeled data structure with columns of potentially different data types. Think of it as a table where each column represents a variable and each row represents an observation.

15.6.1 Creating DataFrames from Dictionaries

You can create a DataFrame from a dictionary, where keys are column labels and values are lists representing data.

```
import pandas as pd
# Create a DataFrame from a dictionary
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],
        'Age': [25, 30, 35, 28],
        'City': ['New York', 'San Francisco', 'Los Angeles', 'Chicago']}
df = pd.DataFrame(data)
print(df)
```

15.6.2 Column Selection

Select a single column using its name or multiple columns by passing a list:

```
print(df['Name'])
print(df[['Name', 'Age']])
```

15.6.3 Accessing Rows

Access rows by position or label using .iloc[] and .loc[]:

```
print(df.iloc[2])   # Third row by position
print(df.loc[1])    # Second row by label
```

15.6.4 Slicing

Slice DataFrames to select specific rows or columns:

```
print(df[['Name', 'Age']])  # Select specific columns
print(df[1:3])              # Select specific rows
```

15.6.5 Finding Unique Elements

Find unique elements in a column:

```
unique_ages = df['Age'].unique()
```

15.6.6 Conditional Filtering

Filter data using conditional expressions:

```
high_above_25 = df[df['Age'] > 25]
```

15.6.7 Saving DataFrames

Save a DataFrame to a CSV file using to_csv():

```
df.to_csv('people_data.csv', index=False)
```

15.7 DataFrame Attributes and Methods

Key attributes and methods for DataFrames include:

- `shape` – Dimensions of the DataFrame.
- `info()` – Summary including data types and null counts.
- `describe()` – Statistical summary of numerical columns.
- `head()`, `tail()` – View first or last n rows.
- `mean()`, `sum()`, `min()`, `max()` – Column-wise summary statistics.
- `sort_values()` – Sort DataFrame by columns.
- `groupby()` – Group data for aggregation.
- `fillna()`, `drop()`, `rename()` – Handle missing data, drop, or rename columns.
- `apply()` – Apply custom functions.

For more details, refer to the official Pandas documentation.

15.8 Conclusion

Mastering Pandas Series and DataFrames is essential for effective data analysis in Python. Series handle one-dimensional labeled data, while DataFrames provide a flexible, table-like structure for two-dimensional data.

By using Pandas, you can efficiently clean, explore, and analyze datasets. Practice with real data and consult Pandas documentation to deepen your understanding and enhance your data analysis skills.

16 Beginner's Guide to NumPy

16.1 Objectives

In this reading, we will learn:

- Basics of NumPy
- How to create NumPy arrays
- Array attributes and indexing
- Basic operations like addition and multiplication

16.2 What is NumPy?

NumPy, short for **Numerical Python**, is a fundamental library for numerical and scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a wide range of mathematical functions to operate on these arrays.

NumPy serves as the foundation for many data science and machine learning libraries, making it an essential tool for data analysis and scientific research.

16.2.1 Key Aspects of NumPy in Python

- **Efficient Data Structures:** NumPy arrays are faster and more memory-efficient than Python lists.
- **Multi-Dimensional Arrays:** Enables representation of matrices and tensors, useful in scientific computing.
- **Element-wise Operations:** Simplifies mathematical operations on entire datasets.
- **Random Number Generation:** Offers tools for simulations and statistical analysis.
- **Integration with Other Libraries:** Works seamlessly with libraries like SciPy, Pandas, and Matplotlib.
- **Performance Optimization:** Built using C and Fortran for high-speed computation.

16.3 Installation

To install NumPy, use the following command:

```
pip install numpy
```

16.4 Creating NumPy Arrays

You can create NumPy arrays from Python lists. Arrays can be one-dimensional or multi-dimensional.

16.4.1 Creating a 1D Array

```
import numpy as np

# Creating a 1D array
arr_1d = np.array([1, 2, 3, 4, 5])
```

Here, a one-dimensional array `arr_1d` is created from a Python list. It contains five elements: 1, 2, 3, 4, and 5.

16.4.2 Creating a 2D Array

```
import numpy as np

# Creating a 2D array
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

This creates a two-dimensional array `arr_2d` representing a 3x3 matrix with elements from 1 to 9.

16.5 Array Attributes

NumPy arrays provide several useful attributes:

```
print(arr_2d.ndim)   # Number of dimensions
print(arr_2d.shape)  # Shape (rows, columns)
print(arr_2d.size)   # Total number of elements
```

Output:

```
2
(3, 3)
9
```

16.6 Indexing and Slicing

Access array elements using indexing and slicing:

```
# Indexing and slicing
print(arr_1d[2])       # Access the 3rd element
print(arr_2d[1, 2])    # Access element at 2nd row, 3rd column
print(arr_2d[1])       # Access the 2nd row
print(arr_2d[:, 1])    # Access the 2nd column
```

16.7 Basic Operations

NumPy allows element-wise operations such as addition, subtraction, multiplication, and division.

16.7.1 Array Addition

```
array1 = np.array([1, 2, 3])
array2 = np.array([4, 5, 6])
result = array1 + array2
print(result)  # [5 7 9]
```

16.7.2 Scalar Multiplication

```
array = np.array([1, 2, 3])
result = array * 2
print(result)  # [2 4 6]
```

16.7.3 Element-wise Multiplication (Hadamard Product)

```
array1 = np.array([1, 2, 3])
array2 = np.array([4, 5, 6])
result = array1 * array2
print(result)  # [4 10 18]
```

16.7.4 Matrix Multiplication

```
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])
result = np.dot(matrix1, matrix2)
print(result)
# [[19 22]
#  [43 50]]
```

16.8 Operations with NumPy

Operation	Description	Example
Array Creation	Creating a NumPy array	arr = np.array([1, 2, 3, 4, 5])
Element-Wise Arithmetic	Element-wise addition, subtraction, etc.	result = arr1 + arr2
Scalar Arithmetic	Addition, subtraction with scalars	result = arr * 2
Element-Wise Functions	Applying functions to each element	result = np.sqrt(arr)

Sum and Mean	Calculating sum and mean	total = np.sum(arr) average = np.mean(arr)
Maximum and Minimum Values	Finding max and min	max_val = np.max(arr) min_val = np.min(arr)
Reshaping	Changing array shape	reshaped_arr = arr.reshape(2, 3)
Transposition	Transposing arrays	transposed_arr = arr.T
Matrix Multiplication	Performing matrix multiplication	result = np.dot(matrix1, matrix2)

16.9 Conclusion

NumPy is a powerful and essential library for numerical and scientific computing in Python. It simplifies data handling with efficient arrays and mathematical operations. This guide introduced NumPy basics, array creation, indexing, and core operations. To explore further, visit numpy.org for more tutorials and examples.

17 Some Context on APIs

17.1 What are APIs?

APIs, or **Application Programming Interfaces**, are a crucial part of software development. They enable developers to build new applications by utilizing existing functionalities from other systems. APIs define how software components interact, facilitating communication between different products and services without requiring direct implementation.

17.2 Importance of APIs

APIs are essential for developers because they allow access to data and functionality from other systems, saving both time and resources.

17.2.1 Benefits of APIs

- Enable integration of applications into existing architectures.
- Facilitate communication between products and services without direct implementation.
- Allow developers to create new applications leveraging existing systems.
- Streamline the engineering and development process.

APIs are widely used across various domains, including social media, e-commerce, web, mobile, and desktop applications.

17.3 Applications of APIs

APIs serve multiple purposes across different industries and platforms.

17.3.1 Social Media Platforms

Platforms like **Facebook**, **Twitter**, and **Instagram** use APIs to let developers access their data and functionality. This allows the creation of apps that enhance user experience and interaction with these platforms.

17.3.2 E-Commerce Websites

E-commerce giants such as **Amazon** and **eBay** use APIs to provide access to product catalogs and transactional data. Developers can build applications that interact with these platforms for enhanced shopping features.

17.3.3 Weather Applications

Weather services like **AccuWeather** and **The Weather Channel** use APIs to distribute real-time weather data. Developers can integrate this data to provide users with up-to-date weather information.

17.3.4 Maps and Navigation Applications

Applications like **Google Maps** and **Waze** use APIs to access geolocation data and navigation details. Developers can integrate these APIs to offer directions, traffic updates, and location-based services.

17.3.5 Payment Gateways

Payment services such as **PayPal** and **Stripe** provide APIs that allow secure payment processing. Developers can integrate these APIs into their apps to handle transactions efficiently and safely.

17.3.6 Messaging Applications

Messaging services like **WhatsApp** and **Facebook Messenger** use APIs to provide access to their messaging capabilities. Developers can use these APIs to build apps that extend messaging functionalities.

17.4 Conclusion

APIs are a fundamental part of modern software development. They enable access to external data and services, enhance functionality, and promote efficiency in development. By leveraging APIs, developers can create powerful, interconnected applications across diverse domains.

18 Module 4 Summary: Working with Data in Python

18.1 File Handling in Python

Python provides the open() function to read and write files, allowing developers to access and manipulate file contents effectively.

18.1.1 File Modes

- **r (read):** Opens the file for reading.
- **w (write):** Opens the file for writing, overwriting existing content.
- **a (append):** Opens the file for appending data without deleting existing content.

18.1.2 Key Functions and Concepts

- open() function: Used to open files for reading or writing.
- with open() statement: Ensures files are properly opened and closed.
- "\n" character: Starts a new line within a file.
- Various methods exist to print and process lines from file attributes.

18.2 Pandas: Data Manipulation and Analysis

Pandas is a powerful Python library for data manipulation and analysis, offering data structures and tools for working with structured data such as DataFrames and Series.

18.2.1 Importing and Aliasing

- Import Pandas using the command: import pandas as pd.
- The as keyword is used to provide a shorter alias for easier access.

18.2.2 Working with DataFrames

- **DataFrame:** A two-dimensional structure consisting of rows and columns.
- You can create new DataFrames using columns from existing ones.
- DataFrames allow reading, manipulating, and saving data in multiple formats.
- The unique() method identifies unique elements in a DataFrame column.
- Inequality operators with df assign Boolean values for conditional filtering.
- You can create new DataFrames derived from existing ones containing filtered values.

18.3 NumPy: Numerical and Matrix Operations

NumPy is a foundational Python library for numerical computations and matrix operations. It provides multidimensional array objects and mathematical functions for efficient data handling.

18.3.1 Key Concepts

- **NumPy as a Foundation:** Pandas is built on top of NumPy.

- **Array (ndarray):** A fixed-size collection of elements of the same data type.
- **1D Array:** A linear sequence of elements optimized for numerical operations.
- Elements are accessed via indexing.

18.3.2 Array Attributes and Methods

- dtype: Returns the data type of array elements.
- size: Returns the total number of elements in the array.
- ndim: Returns the number of array dimensions.
- Indexing and slicing are used to access specific elements or ranges.

18.4 Vector and Matrix Operations in NumPy

NumPy supports fast mathematical operations on vectors and matrices.

18.4.1 Vector Operations

- **Vector Addition:** Adds corresponding elements of arrays.
- **Vector Subtraction:** Replaces the addition sign with a minus sign.
- **Scalar Multiplication:** Multiplies each element of an array by a scalar value.
- **Hadamard Product:** Element-wise multiplication of two arrays with the same shape.
- **Dot Product:** Computes the sum of element-wise products of two arrays, often used in vector and matrix operations.

18.4.2 Visualization with Matplotlib

NumPy often works alongside **Matplotlib** to visualize numerical data from arrays through graphs and charts.

18.5 Two-Dimensional Arrays in NumPy

- A **2D NumPy array** represents data in rows and columns, similar to a matrix or table.
- The shape attribute reveals the number of rows and columns in the array.
- The size attribute gives the total number of elements.
- Elements can be accessed using rectangular indexing.
- Scalars can be used to multiply all elements within an array.

18.6 Summary

- Python enables efficient file handling through the use of open() and related methods.
- Pandas enables powerful data manipulation through DataFrames.
- NumPy provides efficient numerical computation with arrays and matrix operations.
- Combined with Matplotlib, these tools allow comprehensive data processing, analysis, and visualization in Python.

Module 5

19 Application Program Interfaces (APIs)

19.1 Introduction

In this section, we will explore **Application Program Interfaces (APIs)**. Specifically, we will discuss what an API is, API libraries, and REST APIs — including requests, responses, and an example using **PyCoinGecko**.

19.2 What is an API?

An **API** allows two pieces of software to communicate with each other through inputs and outputs.

For example:

- You have a program and some data.
- APIs enable communication between your program and other software components.
- Like a function, you don't need to understand how the API works internally — you only need to know its inputs and outputs.

19.3 APIs in Python

19.3.1 Pandas API

Pandas is an example of a software package that provides an API. Although much of Pandas isn't written in Python, it allows Python code to communicate with underlying components.

- When you create a **dictionary** and pass it to the **DataFrame constructor**, this is considered creating an **instance** in API terminology.
- The data is passed to the Pandas API, and you interact with it through the **DataFrame** object.
- For example:
  - `head()` → Displays the first few rows of data.
  - `mean()` → Calculates and returns mean values.

19.4 REST APIs

REST APIs (Representational State Transfer) are another type of API that allow communication over the **internet**, providing access to services like storage, data, and AI algorithms.

19.4.1 REST API Terminology

- **Client:** The program or code making the request.
- **Resource:** The web service being accessed.
- **Endpoint:** The URL through which the client accesses the resource.
- **Request:** The message sent by the client to the server.

- **Response:** The message sent back from the server to the client.

19.4.2 HTTP and JSON Communication

REST APIs typically use **HTTP** for communication:

- Requests and responses are sent as **HTTP messages**.
- These messages often contain **JSON files** with instructions or data.
- The web service performs the requested operation and sends back a **JSON response**.

19.5 Example: PyCoinGecko API

Cryptocurrency data is ideal for APIs since it updates frequently. We will use the **PyCoinGecko** Python client/wrapper for the **CoinGecko API**, which updates every minute.

19.5.1 Steps to Use PyCoinGecko

1. **Install and import** the library.
2. **Create a client object** to connect to the API.
3. **Request data** using a function call.

Example: Retrieving Bitcoin data in USD for the past 30 days.

- The API returns a **JSON** response as a Python dictionary containing:
  - price
  - market_cap
  - total_volumes
- Each includes a UNIX timestamp and a value.

19.5.2 Data Conversion and Processing

- Select only the **price** data using the key price.
- Convert the nested list to a **DataFrame** with columns 'timestamp' and 'price'.
- Use the **pandas function to_datetime()** to make timestamps readable:
  - Convert the timestamp column (unit: milliseconds) into human-readable **date** values.

19.5.3 Candlestick Chart Creation

To visualize daily price data:

1. **Group by date** to calculate the minimum, maximum, first, and last prices of each day.
2. Use **Plotly** to create a **candlestick chart**.
3. Open the generated HTML file and click **"Trust HTML"** to view the chart.

19.6 Summary

- APIs allow communication between different software components.
- **REST APIs** enable communication over the web using HTTP and JSON.

- **PyCoinGecko** simplifies cryptocurrency data retrieval and analysis.
- Using Pandas and Plotly, we can convert and visualize API data effectively.

20 HTTP Protocol

20.1 Introduction

In this section, we will discuss the **HTTP protocol** — the foundation of communication on the web. Specifically, we will cover the following topics:

- Uniform Resource Locator (URL)
- HTTP Request and Response

20.2 Overview of HTTP

The **HTTP protocol (Hypertext Transfer Protocol)** is a general protocol used for transferring information through the web. It forms the basis of communication for **REST APIs**, which function by sending **requests** and receiving **responses** via HTTP messages.

When you, the **client**, use a web page, your browser sends an **HTTP request** to the **server** where the page is hosted. The server then searches for the requested resource, typically `index.html` by default.

If the request is successful, the server responds with an **HTTP response** that contains the requested object along with metadata such as:

- The type of resource
- The length of the resource
- Other relevant information

20.2.1 Example: Web Server Resources

A typical web server contains various resources such as:

- HTML files
- PNG images
- Text files

When a request is made, the server sends the requested resource (e.g., one of the files) back to the client.

20.3 Uniform Resource Locator (URL)

A **Uniform Resource Locator (URL)** is the standard way to locate resources on the web. A URL can be divided into three main components:

1. **Scheme** – Defines the protocol (e.g., `http://`).
2. **Internet Address (Base URL)** – Indicates the domain or location, such as `www.ibm.com` or `www.gitlab.com`.

3. **Route** – Specifies the path to the resource on the server, for example: /images/IDSNlogo.png.

20.4 HTTP Request and Response

HTTP communication involves **requests** from the client and **responses** from the server.

20.4.1 Request Message

An HTTP **request message** includes the following components:

- **Start Line:** Specifies the HTTP method (e.g., GET) and the resource requested (e.g., index.html).
- **Request Header:** Contains additional information about the request. In a GET request, this header may often be empty.
- **Request Body:** Contains data sent to the server (used in methods like POST).

20.4.2 Response Message

An HTTP **response message** includes:

- **Start Line:** Contains the HTTP version, status code, and a descriptive phrase.
  - Example: HTTP/1.0 200 OK indicates success.
- **Response Header:** Provides additional metadata about the response.
- **Response Body:** Contains the requested data (e.g., an HTML document).

20.5 HTTP Status Codes

HTTP status codes indicate the result of the request. They are grouped by their prefix:

Status Code Range	Type of Response	Example Code	Description
100s	Informational	100	Request received, continuing process
200s	Successful	200	The request has succeeded
400s	Client Error	401	Unauthorized request
500s	Server Error	501	Not implemented

20.6 HTTP Methods

An **HTTP method** defines the action the client wants the server to perform. Common HTTP methods include:

- **GET** – Retrieve data from the server.
- **POST** – Send data to the server.
- **PUT** – Update an existing resource.

- **DELETE** – Remove a resource from the server.

20.7 Summary

- The **HTTP protocol** enables data exchange between clients and servers across the web.
- A **URL** identifies the resource being accessed.
- **Requests** and **responses** are the core of HTTP communication.
- **Status codes** help identify the result of the request.
- HTTP **methods** define the type of action performed during communication.

In the next section, we will use **Python** to apply the GET method for retrieving data from a server and the POST method for sending data to a server.

21 Working with the HTTP Protocol using the Requests Library

21.1 Introduction

In this section, we will explore the **HTTP protocol** in Python using the requests library — a popular and easy-to-use library for handling HTTP requests. We will cover the following topics:

- Overview of the Requests library
- GET Requests
- POST Requests
- Comparison between GET and POST

21.2 Overview of the Requests Library

The **requests** module is one of several Python libraries (along with httplib and urllib) that work with the HTTP protocol. It allows developers to send **HTTP/1.1 requests** effortlessly.

To start using it, import the library as follows:

```
import requests
```

You can make a **GET request** to a website such as www.ibm.com using:

```
r = requests.get('https://www.ibm.com')
```

The response is stored in an object named r, which contains details about the request and the response.

21.3 Exploring the Response Object

The response object includes various attributes and methods for retrieving information:

21.3.1 Checking Status Code

r.status_code

- Returns the HTTP status code (e.g., 200 for success).

21.3.2 Viewing Request Headers

You can access the headers of the request using:

r.request.headers

21.3.3 Viewing Request Body

r.request.body

- For a **GET** request, the body will be None since no data is sent in the body.

21.3.4 Viewing Response Headers

r.headers

- Returns a Python dictionary containing HTTP response headers.

Examples of commonly used keys:

- **Date** – The date and time when the response was sent.
- **Content-Type** – The type of data (e.g., text/html, application/json).

21.3.5 Checking Encoding and Response Text

- To check encoding:

r.encoding

- To display the HTML content:

r.text[:100]

This displays the first 100 characters of the response body.

21.4 GET Requests with Parameters

The **GET** method can also retrieve data from APIs using **query parameters**.

For this example, we will use the test website httpbin.org, a simple HTTP request and response service.

A GET request URL may look like this:

https://httpbin.org/get?name=Joseph&id=123

21.4.1 Structure of a Query String

- Begins with a **?** followed by key-value pairs.
- Each pair is separated by an **=**.
- Multiple pairs are separated by an **&**.

Example:

Parameter	Value
name	Joseph
id	123

21.4.2 Sending GET Request in Python

```
import requests
payload = {'name': 'Joseph', 'id': '123'}
r = requests.get('https://httpbin.org/get', params=payload)
```

You can print out the full URL and view details:

```
print(r.url)  # Shows the URL with parameters
print(r.request.body)  # None, since data is in the URL
print(r.status_code)  # 200 (OK)
```

To view the response as JSON:

```
r.json()
```

This converts the JSON response into a Python dictionary. The key `args` contains the parameters and values sent in the query string.

21.5 POST Requests

A **POST** request sends data to a server in the **request body**, not in the URL.

To send a POST request, modify the route to `/post`:

```
payload = {'name': 'Joseph', 'id': '123'}
r = requests.post('https://httpbin.org/post', data=payload)
```

21.5.1 Differences Between GET and POST

Feature	GET	POST
Data Sent	In URL as query string	In request body
Visibility	Visible in URL	Hidden from URL
Common Use	Retrieve data	Send or submit data

21.5.2 Comparing URLs and Bodies

- The **POST** request URL does **not** contain name-value pairs.
- Only the POST request contains a body.
- You can access the data using the key `form` in the response JSON.

21.6 Summary

- The **requests** library simplifies working with HTTP in Python.
- **GET requests** retrieve data and send parameters via the URL.
- **POST requests** send data through the request body.
- You can access headers, status codes, and content using the response object.
- `httpbin.org` is a helpful site for testing and learning about HTTP requests.

22 Web Scraping and HTML Basics

22.1 Objectives

After completing this reading, you will be able to:

- Explain key concepts related to HTML structure and tag composition.
- Explore the concept of HTML document trees.
- Familiarize yourself with HTML tables.
- Gain insight into web scraping using Python and BeautifulSoup.

22.2 Introduction to Web Scraping

Web scraping, also known as *web harvesting* or *web data extraction*, is the process of extracting information from websites or web pages using automation. It is widely used for:

- Data analysis and mining
- Price comparison
- Content aggregation
- Market research

22.3 How Web Scraping Works

22.3.1 HTTP Request

The process starts with an **HTTP request**, where a web scraper sends a request to a URL—similar to how a browser retrieves a webpage. Most scrapers use the **HTTP GET** method to access web content.

22.3.2 Web Page Retrieval

The server responds with the **HTML content** of the requested webpage, including visible text, images, and the underlying structure that defines its layout.

22.3.3 HTML Parsing

Once received, the HTML content is **parsed** into components (tags, attributes, and text). In Python, the **BeautifulSoup** library is used to parse and navigate this structure.

22.3.4 Data Extraction

After parsing, scrapers identify and extract the required data—such as text, links, images, or tables—by searching for relevant HTML tags and attributes.

22.3.5 Data Transformation

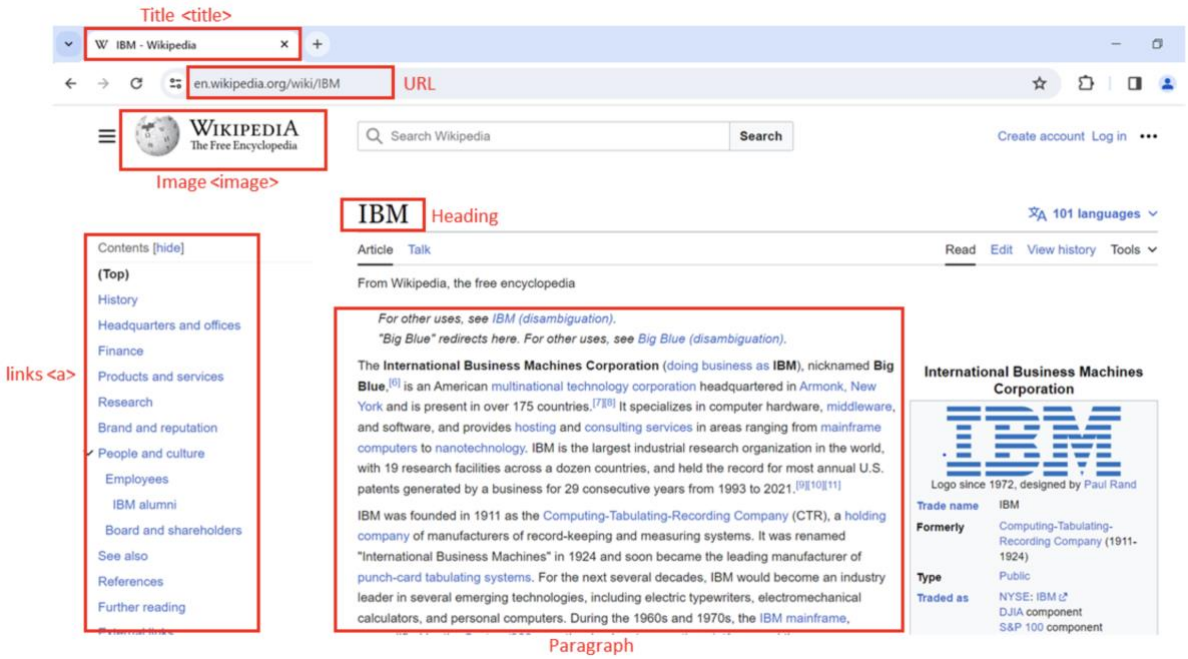
Extracted data is often cleaned or transformed by removing unnecessary HTML tags or formatting inconsistencies before use.

22.3.6 Storage

The cleaned data is then stored in formats like **CSV**, **JSON**, or databases depending on the project requirements.

22.3.7 Automation

Automation tools and scripts can perform repeated scraping across multiple pages or dynamic websites. This is especially helpful when sites frequently update content.



22.4 Document Tree

22.4.1 HTML Structure

HTML (Hypertext Markup Language) forms the backbone of web pages. Its structure consists of nested tags:

- <html>: Root element of the HTML page.
- <head>: Contains metadata about the page.
- <body>: Displays the main content.
- <h3>: Heading level 3 tag, used for titles or subheadings.
- : Paragraph tag, used to hold text content.

22.4.2 Composition of an HTML Tag

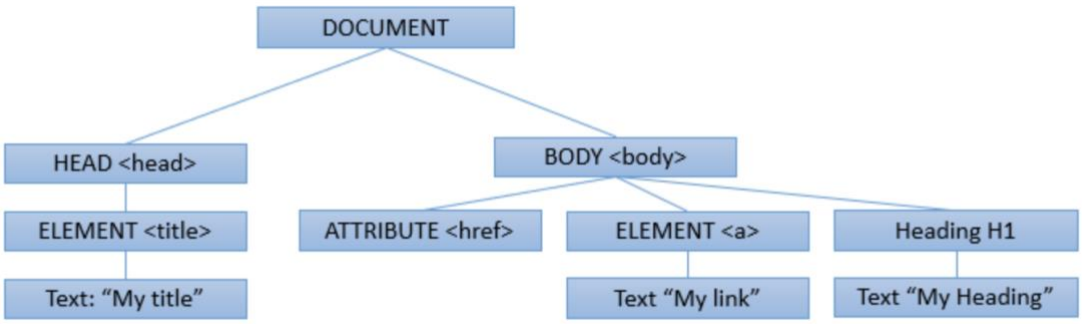
HTML tags define how content is structured and can contain attributes for additional information:

- Tags include an **opening** and **closing** tag (e.g., and).
- Tags may have **attributes** like href or src that provide extra details.

22.4.3 HTML Document Tree

An HTML document can be visualized as a **tree structure** where each tag is a node:

- Tags may contain other tags (children) or text.
- Tags within the same parent are siblings.
- Example: <html> contains <head> and <body> as children, and both are siblings.

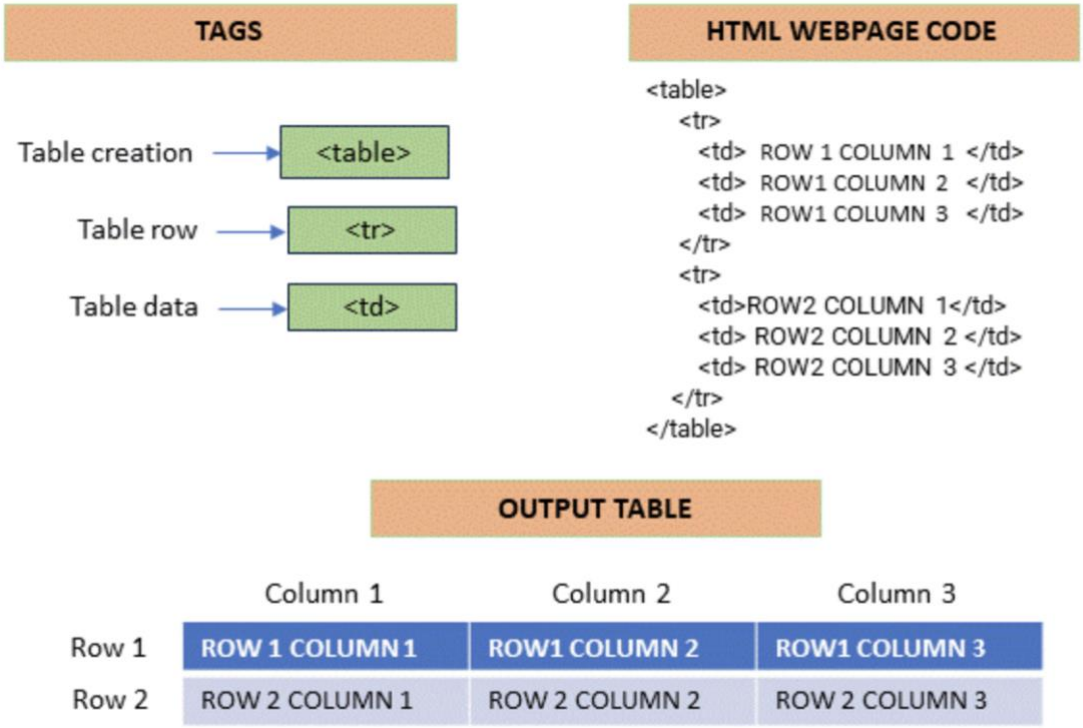


22.5 HTML Tables

HTML tables are used for displaying structured data:

- <table>: Defines the table.
- <tr>: Defines a table row.

- <th>: Table header cell (used for column headings).
- <td>: Table data cell (holds data entries).



22.6 Web Scraping with Python

Web scraping in Python automates data extraction from web pages. It requires two key libraries:

- **Requests** – for sending HTTP requests.
- **BeautifulSoup** – for parsing HTML.

Example:

```
from bs4 import BeautifulSoup
import requests

# Specify the URL
url = 'https://en.wikipedia.org/wiki/IBM'

# Send a GET request
response = requests.get(url)

# Parse the HTML content
soup = BeautifulSoup(response.text, 'html.parser')

# Display the first 500 characters
print(response.text[:500])
```

22.7 Navigating the HTML Structure

BeautifulSoup treats HTML as a **tree-like structure**, making navigation easy. For example:

```
# Find all anchor (<a>) tags
links = soup.find_all('a')

# Print the text of each link
for link in links:
   print(link.text)
```

22.8 Custom Data Extraction

Custom extraction involves locating elements based on tags, attributes, or text content using BeautifulSoup's methods.

22.9 Using BeautifulSoup for HTML Parsing

BeautifulSoup helps you navigate, search, and extract information from HTML efficiently, providing flexibility to target specific sections of a web page.

22.10 Using Pandas read_html for Table Extraction

The **pandas** library provides read_html() to automatically extract HTML tables into DataFrames for analysis:

```
import pandas as pd
url = 'https://en.wikipedia.org/wiki/IBM'
tables = pd.read_html(url)
```

This method is ideal for importing webpage tables into a format suitable for data analysis.

22.11 Conclusion

In this reading, you learned about the basics of **web scraping**, **HTML structure**, and the use of **BeautifulSoup** and **pandas** for data extraction. You now understand how to parse, navigate, and extract data responsibly from web pages.

23 Introduction to HTML for Web Scraping

23.1 Overview

This section reviews **Hypertext Markup Language (HTML)** and its importance in **web scraping**. Understanding HTML allows developers to extract useful data such as real estate listings, player statistics, and much more using Python.

23.2 Objectives

- Review the structure and composition of an HTML document.
- Understand the composition of an HTML tag.

- Explain the concept of HTML document trees.
- Describe HTML tables and their elements.

23.3 Introduction to HTML for Web Scraping

HTML is the language used to structure and display web pages. When performing web scraping, HTML provides the framework from which data is extracted. Python libraries like **BeautifulSoup** allow users to parse and retrieve specific data from HTML code.

Web pages, including those on sites like **Wikipedia**, contain large amounts of valuable information. If you understand HTML, you can easily access and extract this data programmatically.

23.4 Basic HTML Structure

An HTML document is composed of nested elements called **tags**, enclosed in angle brackets < >. Tags define how the content should appear in a browser.

23.4.1 Common HTML Elements

- <!DOCTYPE html> — Declares that the document is written in HTML.
- <html> — The root element that contains all other elements.
- <head> — Contains metadata about the page, such as the title.
- <body> — Contains the visible content displayed in the browser.
- <h3> — Represents a level-3 heading, making text larger and bold.
- <p> — Defines a paragraph of text.

23.4.2 Example

In an example web page showing **National Basketball League** player data:

- The <h3> tags contain player names.
- The <p> tags contain player salaries.

Each piece of information is enclosed within its respective tags, defining its purpose and display.

23.5 Composition of an HTML Tag

HTML tags consist of several components:

23.5.1 Tag Structure

- **Opening Tag:** Defines where an element begins (e.g., <a>)
- **Closing Tag:** Defines where an element ends (e.g.,)
- **Content:** The data or text displayed within the element.
- **Attributes:** Additional information about an element, written as key-value pairs.

23.5.2 Example: Anchor Tag

```
<a href="https://www.ibm.com">IBM</a>
```

- **Tag Name:** a (defines a hyperlink)
- **Attribute:** href (specifies the URL destination)
- **Displayed Content:** IBM

You can think of each tag name as a **class** in Python and each tag instance as an **object**. Real-world web pages often include additional technologies like **CSS** and **JavaScript** for styling and functionality.

23.6 HTML Document Tree

An HTML document can be represented as a **tree structure**, known as the **Document Object Model (DOM)**.

23.6.1 Parent and Child Relationships

- The <html> tag is the **root** element.
- The <head> and <body> tags are **children** of <html>.
- The <title> tag is a **child** of <head>.
- The <h3> and <p> tags are **children** of <body>.
- Tags that share the same parent are called **siblings**.

This hierarchical representation makes it easy to navigate and extract specific parts of a web page using tools like **BeautifulSoup**.

23.7 HTML Tables

HTML tables display data in a structured, grid-like format. They are composed of several key elements:

23.7.1 Table Elements

- <table> — Defines the table.
- <tr> — Represents a table row.
- <th> — Defines a header cell (used for column headings).
- <td> — Defines a data cell.

23.7.2 Example Structure

```
<table>
  <tr>
    <th>Player</th>
    <th>Salary</th>
  </tr>
  <tr>
    <td>John Doe</td>
    <td>$1,000,000</td>
  </tr>
</table>
```

Each <tr> contains multiple <td> elements, representing the individual cells of that row.

23.8 Inspecting and Extracting HTML

Modern browsers allow users to **inspect** web pages to view the underlying HTML structure. This is crucial for identifying which tags contain the data you want to extract.

To inspect a web page:

1. Right-click on a web page.
2. Select **Inspect** or **Inspect Element**.
3. Review the HTML elements in the developer console.

23.9 Summary

- HTML forms the backbone of all web pages.
- Tags define structure, while attributes provide additional information.
- HTML documents are organized as trees, allowing structured navigation.
- Tables store structured data that can be easily extracted.
- Understanding HTML is the foundation for effective web scraping using Python.

24 Web Scraping

24.1 Overview

Now, we'll explore **Web Scraping** — a powerful technique for automatically extracting data from websites using Python. Instead of manually copying information, web scraping allows you to gather large amounts of data in just minutes.

24.2 Objectives

- Define **Web Scraping**.
- Understand the role of **BeautifulSoup Objects**.
- Apply the **find_all()** method.
- Perform basic **web scraping** operations on a website.

24.3 Introduction to Web Scraping

Imagine you want to analyze hundreds of data points to find the best players on a sports team. Manually copying and pasting information into spreadsheets would take hours — or even days. **Web scraping** automates this process, extracting information from websites within minutes using a few lines of Python code.

To begin, we need two essential Python modules:

- **Requests** — to download web pages.
- **BeautifulSoup** — to parse and extract data from HTML.

24.4 BeautifulSoup Object

Let's say you want to find player names and salaries from a National Basketball League webpage.

24.4.1 Creating a BeautifulSoup Object

1. Import the BeautifulSoup library.
2. Store the webpage HTML in a variable, e.g., HTML.
3. Pass it to the BeautifulSoup() constructor to create a soup object.

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(HTML, 'html.parser')
```

The **BeautifulSoup object** represents the HTML document as a **nested tree structure**, making it easy to navigate and search.

24.4.2 Tag Objects

Each **tag object** corresponds to an HTML tag. For example, <title> or <h3>. If multiple tags have the same name, BeautifulSoup selects the first occurrence by default.

In an example with **Lebron James**, the player's name is enclosed within a bold (<b>) tag. Using the **tree representation**, you can navigate to child elements or move between **parent** and **sibling** elements.

24.4.2.1 Navigating the Tree

- **Child Elements:** Access using .contents or .children.
- **Parent Element:** Access using .parent.
- **Siblings:** Access using .next_sibling or .previous_sibling.

24.4.3 Accessing Attributes and Text

Each tag may contain attributes, which can be accessed like dictionary key-value pairs.

```
tag['href']  # Access the href attribute
```

The text within a tag can be retrieved using .string or .text, returning a **NavigableString** — similar to a normal Python string but with BeautifulSoup functionality.

24.5 The find_all() Method

The **find_all()** method filters HTML elements based on tag names, attributes, or text.

24.5.1 Example: Extracting Table Data

Suppose we have a list of pizza places in an HTML table.

1. Create a BeautifulSoup object named table.
2. Use find_all('tr') to find all table rows.

```
table_rows = table.find_all('tr')
```

Each element in table_rows represents a row in the table — including the header row.

3. Iterate through the rows to extract each table cell.

```
for row in table_rows:
   cells = row.find_all('td')
   for cell in cells:
      print(cell.text)
```

Each iteration retrieves cell data from the table, enabling structured extraction.

24.6 Scraping a Webpage with Requests and BeautifulSoup

To apply web scraping on a real webpage, we also use the **Requests** library.

24.6.1 Steps:

1. **Import the Modules:**

```
import requests
from bs4 import BeautifulSoup
```

2. **Download the Webpage:**

```
page = requests.get('https://example.com').text
```

3. **Create a BeautifulSoup Object:**

```
soup = BeautifulSoup(page, 'html.parser')
```

4. **Scrape the Desired Content:**
   Navigate and extract the desired information from the soup using BeautifulSoup methods.

24.7 Summary

- **Web Scraping** automates data extraction from websites using Python.
- **Requests** retrieves the webpage's HTML, while **BeautifulSoup** parses and organizes it.
- **BeautifulSoup Objects** represent HTML as a tree structure for easy navigation.
- The **find_all()** method is used to filter and extract specific elements.
- Together, these tools allow you to collect and analyze data efficiently from online sources.

25 Web Scraping: A Key Tool in Data Science

25.1 Introduction

Web scraping, also known as **web harvesting** or **web data extraction**, is a technique used to extract large amounts of data from websites. The data on websites is often **unstructured**, and web scraping enables us to convert it into a **structured** and analyzable form.

25.2 Importance of Web Scraping in Data Science

In data science, **web scraping** plays a vital role in data collection and analysis. It is used for multiple purposes:

25.2.1 Data Collection

Web scraping is a primary method for gathering data from the internet. This collected data can then be used for **analysis, research, and insights**.

25.2.2 Real-time Applications

It is widely used for **real-time** use cases, such as **weather updates**, **price comparison**, and **news aggregation**.

25.2.3 Machine Learning

Machine learning models require large datasets for training. Web scraping provides the raw data used to build these models.

25.3 Web Scraping with Python

Python offers several libraries to perform efficient and scalable web scraping.

25.3.1 BeautifulSoup

BeautifulSoup is a Python library used to extract data from **HTML** and **XML** documents. It parses page source code and organizes it in a tree structure, making data extraction simple and readable.

```
from bs4 import BeautifulSoup
import requests

URL = "http://www.example.com"
page = requests.get(URL)
soup = BeautifulSoup(page.content, "html.parser")
```

25.3.2 Scrapy

Scrapy is an open-source web crawling and scraping framework for Python. It is designed for large-scale data extraction.

```
import scrapy

class QuotesSpider(scrapy.Spider):
    name = "quotes"
    start_urls = ['http://quotes.toscrape.com/tag/humor/']

    def parse(self, response):
        for quote in response.css('div.quote'):
            yield {'quote': quote.css('span.text::text').get()}
```

25.3.3 Selenium

Selenium is a powerful tool for **automating web browsers**. It is often used for scraping dynamic content that requires JavaScript execution.

```
from selenium import webdriver

driver = webdriver.Firefox()
driver.get("http://www.example.com")
```

25.4 Applications of Web Scraping

Web scraping is used across various industries and has many real-world applications:

25.4.1 Price Comparison

Web scraping tools like **ParseHub** gather product data from e-commerce sites and compare prices across platforms.

25.4.2 Email Address Gathering

Companies that use **email marketing** employ scraping techniques to collect email addresses for targeted campaigns.

25.4.3 Social Media Scraping

It is used to extract data from social media platforms like **Twitter** or **Instagram** to analyze trends, user sentiment, and engagement.

25.5 Conclusion

Web scraping is a critical skill in the data science toolkit. It transforms the web into a rich source of data for analysis and innovation. However, it must be practiced **ethically** and **responsibly**, respecting each website's **terms of use** and **robots.txt** policies.

26 Web Scraping Tables using Pandas

26.1 Introduction

The **Pandas** library in Python contains a function called read_html() that can be used to **extract tabular information** directly from web pages. This function allows users to quickly collect and analyze data presented in HTML tables.

26.2 Example: Extracting Largest Banks by Market Capitalization

To understand how read_html() works, let's extract the **list of the largest banks in the world by market capitalization** from the following URL:

URL = 'https://en.wikipedia.org/wiki/List_of_largest_banks'

We can use the pandas.read_html() function to extract all the tables from this web page:

```
import pandas as pd
URL = 'https://en.wikipedia.org/wiki/List_of_largest_banks'
tables = pd.read_html(URL)
df = tables[0]
print(df)
```

This code extracts all tables from the web page and stores the **first table** (index 0) in a DataFrame called df. The extracted table displays the list of the largest banks by market capitalization.

	Rank	Bank name	Market cap(US$ billion)
0	1	JPMorgan Chase	419.25
1	2	Bank of America	231.52
2	3	Industrial and Commercial Bank of China	194.56
3	4	Agricultural Bank of China	160.68
4	5	HDFC Bank	157.91
5	6	Wells Fargo	155.87
6	7	HSBC Holdings PLC	148.90
7	8	Morgan Stanley	140.83
8	9	China Construction Bank	139.82
9	10	Bank of China	136.81

Note: The web page is live and may be updated over time. The process of extraction remains the same regardless of future updates.

26.3 Limitations of Using `read_html()`

Although convenient, this method has certain **limitations**:

26.3.1 1. Non-Visible Tables

Some web pages store content in **table tags** that do not visually appear as tables. For example:

URL = 'https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)'

The page contains several tables, including ones that represent **images** stored in tabular form.

List of countries by GDP (nominal)

75 languages

Article Talk Read View source View history Tools

From Wikipedia, the free encyclopedia

For countries by GDP based on purchasing power parity, see List of countries by GDP (PPP).
For countries by GDP per capita, see List of countries by GDP (nominal) per capita.

Gross domestic product (GDP) is the market value of all final goods and services from a nation in a given year.[1] Countries are sorted by nominal GDP estimates from financial and statistical institutions, which are calculated at market or government official exchange rates. Nominal GDP does not take into account differences in the cost of living in different countries, and the results can vary greatly from one year to another based on fluctuations in the exchange rates of the country's currency.[2] Such fluctuations may change a country's ranking from one year to the next, even though they often make little or no difference in the standard of living of its population.[3]

Comparisons of national wealth are also frequently made on the basis of purchasing power parity (PPP), to adjust for differences in the cost of living in different countries. Other metrics, nominal GDP per capita and a corresponding GDP (PPP) per capita are used for comparing national standard of living. On the whole, PPP per capita figures are less spread than nominal GDP per capita figures.[3]

The rankings of national economies over time have changed considerably; the United States surpassed the British Empire's output around 1916,[5] which in turn had surpassed the Qing dynasty in aggregate output decades earlier.[7][8] Since China's transition to a socialist market economy through controlled privatisation and deregulation,[9][10] the country has seen its ranking increase from ninth in 1978, to second in 2010. China's economic growth accelerated during this period and its share of global nominal GDP surged from 2% in 1980 to 18% in 2021.[8][11][11] Among others, India has also experienced an economic boom since the implementation of economic liberalisation in the early 1990s.[12]

The first list includes estimates compiled by the International Monetary Fund's World Economic Outlook, the second list shows the World Bank's data, and the third list includes data compiled by the United Nations Statistics Division. The IMF definitive data for the past year and estimates for the current year are published twice a year in April and October. Non-sovereign entities (the world, continents, and some dependent territories) and states with limited international recognition (such as Kosovo and Taiwan) are included in the list where they appear in the sources.

Largest economies in the world by GDP (nominal) in 2023 according to International Monetary Fund estimates[4][5]

Countries by nominal GDP in 2019[6]

Table 1

Table 2

Table 3

Table

The table initially ranks each country or territory with their latest available estimates, and can be reranked by either of the sources.

The links in the "Country/Territory" row of the following table link to the article on the GDP or the economy of the respective country or territory.

GDP (USD million) by country

	Country/Territory	UN region	IMF[12][13]		World Bank[14]		United Nations[15]	
			Forecast	Year	Estimate	Year	Estimate	Year
	World	—	**104,476,432**	**2023**	**100,662,011**	**2022**	**96,690,006**	**2021**
1	United States	Americas	26,949,643	2023	25,462,700	2022	23,315,081	2021
2	China	Asia	17,700,899	[n 1]2023	17,963,171	[n 2]2022	17,734,131	[n 1]2021
3	Germany	Europe	4,429,838	2023	4,072,192	2022	4,259,935	2021
4	Japan	Asia	4,230,862	2023	4,231,141	2022	4,940,878	2021
5	India	Asia	3,732,224	2023	3,385,090	2022	3,201,471	2021
6	United Kingdom	Europe	3,332,059	2023	3,070,668	2022	3,131,378	2021
7	France	Europe	3,049,016	2023	2,782,905	2022	2,957,880	2021
8	Italy	Europe	2,186,082	2023	2,010,432	2022	2,107,703	2021
9	Brazil	Americas	2,126,809	2023	1,920,096	2022	1,608,981	2021
10	Canada	Americas	2,117,805	2023	2,139,840	2022	1,988,336	2021
11	Russia	Europe	1,862,470	2023	2,240,422	2022	1,778,782	2021
12	Mexico	Americas	1,811,468	2023	1,414,187	2022	1,272,839	2021
13	South Korea	Asia	1,709,232	2023	1,665,246	2022	1,810,968	2021
14	Australia	Oceania	1,687,713	2023	1,675,419	2022	1,734,532	2021
15	Spain	Europe	1,582,054	2023	1,397,509	2022	1,427,381	2021
16	Indonesia	Asia	1,417,387	2023	1,319,100	2022	1,186,093	2021
17	Turkey	Asia	1,154,600	2023	905,988	2022	819,034	2021

26.3.2 2. Extra HTML Elements

The extracted data may include unwanted elements such as **hyperlinks**, **footnotes**, or **symbols**. These are scraped directly by Pandas, which may require **additional data cleaning**.

GDP (USD million) by country

	Country/Territory	UN region	IMF[1][13]		World Bank[14]		United Nations[15]	
			Forecast	Year	Estimate	Year	Estimate	Year
	World	—	**104,476,432**	**2023**	**100,562,011**	**2022**	**96,698,005**	**2021**
1	United States	Americas	26,949,643	2023	25,462,700	2022	23,315,081	2021
2	China	Asia	17,700,899	[n 1]2023	17,963,171	[n 3]2022	17,734,131	[n 1]2021
3	Germany	Europe	4,429,838	2023	4,072,192	2022	4,259,935	2021
4	Japan	Asia	4,230,862	2023	4,231,141	2022	4,940,878	2021
5	India	Asia	3,732,224	2023	3,385,090	2022	3,201,471	2021
6	United Kingdom	Europe	3,332,059	2023	3,070,668	2022	3,131,378	2021
7	France	Europe	3,049,016	2023	2,782,905	2022	2,957,880	2021
8	Italy	Europe	2,186,082	2023	2,010,432	2022	2,107,703	2021
9	Brazil	Americas	2,126,809	2023	1,920,096	2022	1,608,981	2021
10	Canada	Americas	2,117,805	2023	2,139,840	2022	1,988,336	2021
11	Russia	Europe	1,862,470	2023	2,240,422	2022	1,778,782	2021
12	Mexico	Americas	1,811,468	2023	1,414,187	2022	1,272,839	2021
13	South Korea	Asia	1,709,232	2023	1,665,246	2022	1,810,966	2021
14	Australia	Oceania	1,687,713	2023	1,675,419	2022	1,734,532	2021
15	Spain	Europe	1,582,054	2023	1,397,509	2022	1,427,381	2021

26.4 Example: Extracting a Table with Hyperlinks

Let's extract a table from the page listing countries by GDP:

```
import pandas as pd
URL = 'https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)'
tables = pd.read_html(URL)
df = tables[2]  # The required table has index 2
print(df)
```

```
    Country/Territory UN region IMF[1][13]          World Bank[14]          United Nations[15]
    Country/Territory UN region   Forecast    Year      Estimate    Year          Estimate    Year
0               World         –  104476432    2023     100562011    2022          96698005    2021
1       United States  Americas   26949643    2023      25462700    2022          23315081    2021
2               China      Asia   17700899  [n 1]2023     17963171  [n 3]2022       17734131  [n 1]2021
3             Germany    Europe    4429838    2023       4072192    2022           4259935    2021
4               Japan      Asia    4230862    2023       4231141    2022           4940878    2021
..                ...       ...        ...     ...           ...     ...               ...     ...
209             Palau   Oceania        267    2023             –       –               218    2021
210          Kiribati   Oceania        246    2023           223    2022               227    2021
211             Nauru   Oceania        150    2023           151    2022               155    2021
212        Montserrat  Americas          –       –             –       –                72    2021
213            Tuvalu   Oceania         63    2023            60    2022                60    2021
```

The extracted DataFrame contains hyperlink texts and other embedded elements, which need to be cleaned if only pure textual data is desired.

26.5 Comparison with BeautifulSoup

While pandas.read_html() is ideal for **tabular data extraction**, it only works with structured table formats. For **non-tabular or complex HTML data extraction**, the **BeautifulSoup** library remains the default and more flexible option.

26.6 Summary

- pandas.read_html() extracts tabular data directly from web pages into DataFrames.
- It is simple and effective, but may include extra HTML elements like hyperlinks.
- This method works best when the data is well-structured in HTML tables.
- For complex or unstructured data, use **BeautifulSoup** for customized extraction.

27 Working With Different File Formats

27.1 Learning Objectives

- Define different file formats such as **CSV**, **XML**, and **JSON**.
- Write simple programs to read and output data.
- List the Python libraries needed to extract data.

27.2 Introduction

When collecting data, you will encounter many different file formats that need to be read or processed for analysis. Python simplifies this process with its predefined libraries. Before diving into Python, let's review some common file formats.

27.3 Understanding File Formats

Each file has an **extension** at the end of its name, which identifies its format and the type of software required to open it. For example:

- A file named FileExample.csv indicates it is a **CSV** (Comma-Separated Values) file.
- Other common file formats include **JSON** and **XML**.

When working with these file formats in Python, specific libraries help you read and extract data efficiently.

27.4 Using Pandas to Read CSV Files

The first Python library to learn for handling files is **Pandas**. Once imported, Pandas allows you to easily read various file types.

27.4.1 Reading a CSV File

Example:

```
import pandas as pd

file = 'FileExample.csv'
df = pd.read_csv(file)
print(df)
```

In this example, the CSV file had no headers, so the first line of data was automatically set as the header.

27.4.2 Adding Headers to a CSV File

To make the output more organized, you can manually define the headers:

```
df.columns = ['Column1', 'Column2', 'Column3']
print(df)
```

This ensures that the data is neatly organized with proper column names.

27.5 Working with JSON Files

The next file format is **JSON** (JavaScript Object Notation). It stores data in a structured, language-independent format similar to Python dictionaries.

27.5.1 Reading a JSON File

Example:

```
import json

with open('FileExample.json') as file:
    data = json.load(file)
print(data)
```

Here, we import the `json` library, open the file, use the `load()` function to read it, and print the data.

27.6 Working with XML Files

The **XML** (Extensible Markup Language) format is used to store and transport data. Although Pandas cannot read XML directly, Python provides tools to parse it.

27.6.1 Reading an XML File

Example:

```
import xml.etree.ElementTree as ET

tree = ET.parse('FileExample.xml')
root = tree.getroot()

for child in root:
    print(child.tag, child.attrib)
```

In this example, we import the `xml` library, use the `etree` module to parse the XML file, and loop through elements to extract information.

27.6.2 Creating a DataFrame from XML Data

After parsing the XML, you can assign column headers, collect the required data, and append it to a Pandas DataFrame for analysis.

27.7 Summary

In this video, you learned:

- How to recognize different file types (**CSV**, **JSON**, **XML**).
- How to use Python libraries such as **Pandas**, **JSON**, and **XML** to extract and process data.
- How to use **DataFrames** to organize and analyze data effectively.

28 Summary

28.1 Introduction

Simple APIs in Python are application programming interfaces that provide straightforward and easy-to-use methods for interacting with services, libraries, or data, often with minimal configuration or complexity. An API lets two pieces of software communicate with each other.

28.2 Using APIs in Python

Using an API library in Python involves:

1. Importing the library.
2. Calling its functions or methods to make HTTP requests.
3. Parsing responses to access data or services provided by the API.

28.2.1 Pandas API

The Pandas API processes data by communicating with other software components.

28.2.1.1 Creating a Pandas Instance

An instance is created when you form a dictionary and then use the **DataFrame** constructor to create a Pandas object.

28.2.1.2 Common Methods

- **head()** – Displays the mentioned number of rows from the top (default 5) of DataFrames.
- **mean()** – Calculates the mean and returns the values.

28.3 REST APIs and HTTP Communication

REST APIs allow communication over the internet, providing access to resources such as storage, data, and AI algorithms.

28.3.1 HTTP Protocol

The **HTTP (HyperText Transfer Protocol)** transfers data, including web pages and resources, between a client (web browser) and a server on the World Wide Web. It is the foundation for implementing REST APIs.

28.3.2 HTTP Methods

- **GET** – Requests information from the server.
- **POST** – Submits data to the server.
- **PUT** – Updates existing data on the server.
- **DELETE** – Removes data from the server.

28.3.3 HTTP Messages and Responses

HTTP messages transmit data over the internet, typically containing JSON files with operational instructions. These responses include details like resource type, resource length, and other metadata.

28.4 URL Structure

A **Uniform Resource Locator (URL)** identifies and locates resources on the web. It is divided into three parts:

1. **Scheme** – Defines the protocol (e.g., HTTP or HTTPS).
2. **Internet Address/Base URL** – The domain name or IP address.
3. **Route** – The specific path to the resource.

You can modify query results with the **GET** method and obtain multiple parameters (like name, ID, etc.) from a URL using a **Query string**.

28.5 Working with the Requests Library

Requests is a Python library that simplifies sending HTTP/1.1 requests. It allows developers to easily interact with web services using GET, POST, PUT, and DELETE methods.

28.6 Time Series Data and Plotting

When dealing with time series data, the **Pandas time series functions** help process and analyze it. You can visualize daily candlestick data using **Plotly** to plot interactive charts.

28.7 Web Scraping in Python

Web scraping involves extracting and parsing data from websites to gather information for analysis. Libraries like **Beautiful Soup** and **Requests** are commonly used.

28.7.1 Understanding HTML

HTML comprises text enclosed within elements called **tags**, represented inside angular brackets (e.g., <p>,). Web pages often also contain **CSS** and **JavaScript**.

Each HTML document can be viewed as a **tree structure** with nested elements, including strings and tags. HTML tables use elements like <table>, <tr>, <th>, and <td> to organize data.

28.7.2 Extracting Tabular Data

You can extract tabular data from web pages using Pandas' read_html() method.

28.7.3 Beautiful Soup Library

Beautiful Soup is a Python library for parsing and navigating HTML or XML documents.

28.7.3.1 Parsing a Document

Pass an HTML document to the **BeautifulSoup** constructor to create a structured object for navigation and data extraction.

28.7.3.2 Navigable String and find_all Method

- A **NavigableString** behaves like a Python string but supports Beautiful Soup operations.
- The **find_all()** method extracts content based on tag names, attributes, or text. It searches through tag descendants and retrieves all matching elements, returning them as a Python iterable (e.g., list).

28.8 Working with File Formats

File formats define the structure and encoding of stored data. Common examples include:

- .txt for plain text
- .csv for comma-separated values
- .xml for Extensible Markup Language
- .json for JavaScript Object Notation
- .xlsx for Excel spreadsheets

Python supports multiple file formats and provides libraries to handle them efficiently.

28.8.1 Accessing Different File Types

- **CSV**: Use Pandas to read and process CSV files.
- **JSON**: Use Python's json module to parse and handle JSON data.
- **XML**: Use xml.etree.ElementTree for XML file parsing.

The file extension indicates the type of file and the appropriate tool required to open or process it.

28.9 Summary

- APIs enable software systems to communicate effectively.
- REST APIs use HTTP methods like GET, POST, PUT, and DELETE for operations.
- The Requests library allows interaction with APIs easily.
- Web scraping extracts data using Beautiful Soup and Pandas.
- Python supports multiple file formats, including CSV, JSON, and XML, through specialized libraries.